#Binary Search

Agenda:

1. Intro to Searching
2. Search for the element k in sorted array with unique elements
3. Search 1st and last occurence of k in sorted array with repeated elements
4. Find a peak element
5. Local Minima

Q① Find the index of a target k in a sorted array with unique elements. If not found, return -1.

T.C = O(log N)
S.C = O(1)

Sol. **Binary Search CORE CONCEPT:**

① Define Search space: Set a start (low) and end (high) point.

② Check middle: Calculate the middle index and see if it the ans.

③ Decide Direction: If it's not the answer, decide if you need to search the left half or the right half, then repeat.

code →
```
public static int search (int[] arr, int k){
        int low=0, high = arr.length-1;
    while (low <= high){
        int mid = (low+high) /2 ;      // Find the middle
        if (arr[mid] == k) return mid ;  // Found it!
//Search right half  else if (arr[mid] < k) low = mid+1 ;
//Search Left half       else high = mid-1 ;
    }
    return -1 ;        //Not Found
}
```

Q② When an array has repeated elements (Like [5, 5, 5]), a standard search might return any of 5. To find the first or last one, we add an extra check.

#first occurence

code →
```
if (arr[mid] == k && (mid == 0 || arr[mid-1] < k)){
        return mid;
} else if (arr[mid] < k){        #first occurence
        low = mid+1;
} else {
        high = mid-1;
}
```

```
# last occorence
while (l<=h) {
        int mid = (l+h)/2 ;
        if ( A[mid] == B && (mid = A.length -1 || A[mid+1]>B)){
                last=mid ;
                break;
        }
        else if (A[mid]<=B)  l = mid+1 ;
        else  h = mid-1 ;
```

---

(4) Finding the Peak Element → max element of the array

```
int l=0;
int h= A.length -1 ;
while (l<h) {
        int mid= (l+h)/2 ;
        if (A[mid] < A[mid+1]) {      // Increasing slope → peak on right
                l= mid+1 ;
        }
        else {
                h= mid ;              // Decreasing slope → peak on left (including mid)
        }
}
return A[l];                          // l==h → peak index
}
```

---

(5) Local Minima → Find any element that is smaller than its neighbors

↳ code: next page

corner case ; for 1st and last elements, you only check their one neighbour.

Logic: If the element to the left of (mid) is smaller, a local minimum must exist on the left side. If the element to the right is smaller, it's on the right.

Code →

```
Public static int findLocalMinima (int[]A){
        int n = A.length;
        int l=0, r = n-1;
    While (l <=r){
            int mid = (l+r)/2;
    #check if mid is local minima
if ((mid==0 || A[mid-1] > A[mid]) &&
    (mid ==n-1 || A[mid+1] > A[mid])) {
                return A[mid];
        }
    if (mid>0 && A[mid-1] <A[mid]){
            r=mid-1;
        }
        else {
            l = mid+1;
        }
    }
    return n-1;
}
```

Binary Search-2

Q.1 Given an integer sorted arr[], where every element occurs twice except for one, find the unique element. Note: Duplicate elements are adjacent to each other

T.C = $O(\log_2 n)$
S.C = $O(1)$

Agenda:
1. Single no. with sorted array
2. Search in Rotated sorted array
3. Painter's Partition - Find min count of painters
4. Square root

Sol.

- Before the unique element:
  Pairs start at an even index and end at an odd index (eg: 0-1, 2-3 indices)

- After the unique element: Pairs start at an odd index and end at an even index (eg: 9-10, 11, -12)

Code →

```
Public int FindUnique (int[]A) {
    int n=A.length;
    if (n==1) return A[0];

    int low =0, high =n-1;
    while (low <=high){
        int mid =(low + high)/ 2

        // check if mid is the unique element
        if ((mid ==0 || A[mid] != A[mid-1]) &&
            (mid ==n-1 || A[mid] != A[mid+1])) {
                return A[mid];

        }

        # Determine if we are in the "Left" or "Right" side.
        # If mid is even and matches next, or mid is odd and
            matches previous.

        # It means we haven't hit the unique element yet.
        if ((mid %2 ==0 && A[mid] == A[mid+1]) ||
            (mid %2 ==1 && A[mid]== A[mid-1])){
                low=mid+1 ; // Go right

        } else {
            high = mid-1 ; //Go left

        }
    }
    return -1;
}
```

→ Trick: if rotated one half or array is always sorted

(2) Search in Rotated Sorted array.

```
Public int searchRotated (int[]A, int k){
    int low = 0, high = A.length-1;
while(L<=h)     int mid = (low+high)/2
                if (A[mid]==k) return mid;

    # Identify which part is sorted
if (A[low] <= A[mid]) {  //left side is sorted
        if (k >= A[low] && k < A(mid]) ~~high=mid+1~~
                high = mid-1;
        else low = mid+1;
    }
else {
    if (k > A[mid] && k <= A[high]) low=mid+1;
    else high = mid-1;
    }
}
return -1;
}
```

1 3 8 10 12 15 16 18 20

16 18 20 1 3 8 10 12 15

(3) Painter's Partition Problem

Q. Given N Boards with length of each board

(a) A painter takes 1 unit of time to paint 1 unit of length.

(b) A board can only be painted by 1 painter.

(c) A painter can only paint boards placed next to each other (i.e. continoous segment).

Q. Find the minimum count of painters required to paint all the boards in (T) unit of time.

Trick: This is a greedy approch,
Start a painter, keep adding boards until they hit the limit T, then bring in a new painter.

Code →

```
public int minPainters (int[]A, int T){
    int Painters = 1;
    int currentTime = 0;

    for (int Length : A){
        if (Length > T) return -1;   // single board exceeds total time allowed.

        if (currentTime + length <= T)
            currentTime += length;   // same painter continues

        else {
            painters++;   // Bring in a new Painter.
            currentTime = length;   // New painter starts with this board.
        }
    }
    return painters;
}
```

---

(4) **Integer Square Root**

$TC = O(\log_2 N)$ $SC = O(1)$

* without using In-build function

The Problem: Find the floor value of $\sqrt{N}$ (e.g. $\sqrt{50} = 7$)

The Trick: The numbers from 1 to N are sorted. We can binary search for a number x such that $x^2 \le N$ but $(x+1)^2 > N$.

Code →

```
public int sqrt(int n){
    if (n==0 || n==1) return n;
    int lo = 1, hi = n, ans = 0;

    while (lo <= hi){
        int mid = lo + (hi-lo)/2;
        if ((long) mid*mid <= n){
            ans = mid;
            lo = mid+1;
        }
        else{ hi = mid-1;}
    }
    return ans;
}
```

Binary Search-3

Agenda:

1. Painter's Partition
   - find min time to paint
2. Aggresive cows
   - find max count of cows
3. Aggresive cows
   - find max possible min distance

(1) Painter's Partition Problem (Part-2)

↳ Goal: Find the minimum time required to paint all boards using P Painters.

Rule:
- A painter takes 1 unit of time to paint 1 unit of length
- Each board is painted by exactly one painter.
- Painters must paint continuous segments of boards.

```
Public class PrinterSPartition {
    Static boolean canPaint (int[] boards, int k, int Limit) {
        int painters = 1
        int workdone = 0;
        For (int Length : boards) {
            if (length > Limit) return false;
            if (workDone + Length <= limit) {
                workDone += length;
            }
            else {
                painters ++ ;
                workDone = length;
            }
        }
        return painters <= k;
    }

Public Static int FindMinTime (int[] boards, int k) {
        int low = 0
        int high = 0;
    For (int b : boards) {
            low = Math.max (low, b);
            high += b;
        }
    int answer = high;
```

```
while (low <= high) {
        int mid = low + (high-low)/2 ;
    if (canPaint (boards, k, mid)) {
            answer = mid
            high = mid-1;
        }
    else {
            low = mid + 1;
        }
    }return answer;
}
```

② # Aggresive Cows-1 → barn has M stalls and different locations at x-axis

Q. Find the max no. of cows that can be placed if all pairs of cows should at-least be D distance apart.

Sol.

```
Public class Aggresive Cows Q1 {
    Public static int maxCowsFixedDistance (int[] stalls, int D) {
        int count = 1;
        int lastPlaced = stalls[0];

        for (int i = 1; i < stalls.length ; i++) {
            if (stalls[i] - lastPlaced >= D) {
                count++;
                lastPlaced = stalls[i];
            }
        }
        return count;
    }
}
```

TC: $O(N)$

S.C: $O(1)$

③ # Aggresive Cows-2

Q. Find maximum possible minimum distance b/w any 2 pairs of cows if farmer has C cows.

Sol. **Define Range**: Smallest mindist. = 0
largest min dist. = $A[N-1] - A[0]$

Binary Search: Pick a middle dist. (mid) and Check if it's possible to place all C cows with atleast that much space.

Adjust: If you can place C or more cows. this distance works! Try a Larger distance to see if you can do better (L = mid+1).

- If you can't place enough cows, the distance is too large. Try a smaller dist (R = mid-1)

Code →

```
Public class AggresireCowsQ2{
    Static int coontCows(int[] stalls, int dist){
        int coont = 1;
        int lastPos = stalls[0];
        For (int i =1; i < stalls.length; i++){
            if (stalls[i] - lastPos >= dist){
                coont++;
                lastPos = stalls[i];
            }
        }
        return coont;
    }
```

Helper

```
Public static int largestMinDist (int[] stalls, int c)
    Arrays.sort (stalls);
    int lo = 0;
    int hi = stalls [stalls.length-1] - stalls[0];
    int ans = 0;
    While (lo <= hi){
        int mid = lo + (hi - lo)/2;
        if(coontCows (stalls, mid) >= c){
            ans = mid;
            lo = mid+1;
        }
        else{
            hi = mid-1;
        }
    }
    return ans;
}
}
```

main

$$\boxed{\begin{aligned} T.C &= O(N \cdot \log(\text{Range})) \\ S.C &= O(1) \end{aligned}}$$